Tables des matières

10. Scalabilité de la plateforme 2

Introduction 2

10.1 Objectif de scalabilité 2

10.2 Architecture cible "scalable" 2

1. Kestra (orchestration) 2

2. MinIO (stockage S3-like) 3

3. PostgreSQL (base analytique) 3

4. Couche d'exécution / infrastructure 4

5. Observabilité & supervision 4

10.3 Comment il faut le faire (approche par étapes) 4

Étape 1 – Séparer les rôles sur plusieurs services 4

Étape 2 – Rendre chaque brique "HA" 4

Étape 3 – Industrialiser le déploiement 4

Étape 4 – Indicateurs de capacité 5

10.4 Coût de la scalabilité 5

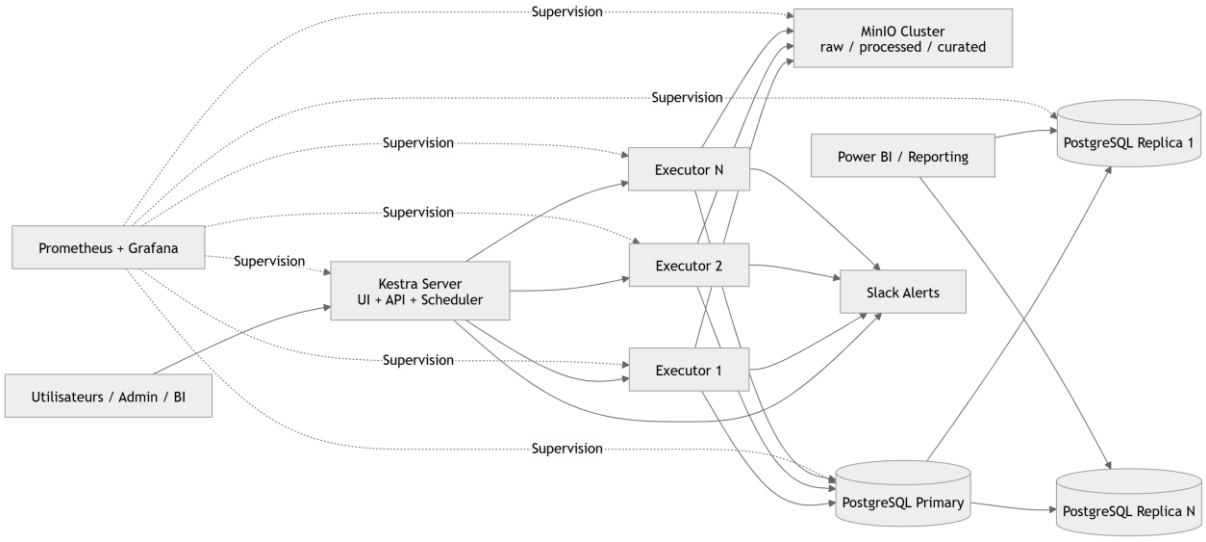
Le tableau ci-dessous présente une estimation des coûts mensuels associés à une architecture scalable. 5

Conclusion 5

10. Scalabilité de la plateforme

Introduction

Cette section décrit les mécanismes de scalabilité de la plateforme de données. Bien que ces éléments ne soient pas implémentés dans le cadre du POC, ils permettent d'anticiper une mise en production dans un contexte de montée en charge.



Lecture de l'architecture

- **Kestra Server** centralise l'interface, l'API et le scheduler.
- Les **executors** exécutent les flows en parallèle.
- **MinIO** passe en **cluster distribué** pour éviter le point de panne unique.
- **PostgreSQL** est séparé en :
  - **Primary** pour les écritures du pipeline
  - **Replicas** pour les lectures BI et reporting
- **Slack** reçoit les alertes d'exécution et d'échec.
- **Prometheus + Grafana** assurent la supervision technique.

10.1 Objectif de scalabilité

L'objectif est d'assurer la continuité et la performance du pipeline dans un contexte de croissance des données et des usages.

- Assurer la montée en charge : plus d'employés, plus d'activités, plus de fichiers.
- Garantir la disponibilité : éviter qu'un seul serveur Kestra / Postgres / MinIO soit un point de panne.
- Maîtriser les coûts : payer (ou dimensionner) en fonction du volume réel.

10.2 Architecture cible "scalable"

1. Kestra (orchestration)

L’architecture cible repose sur un mode distribué permettant de paralléliser l’exécution des traitements.

Aujourd’hui : 1 instance Kestra “standalone” dans une VM.

Cible scalable :

- 1 nœud “server” (UI, API, scheduler)
- N nœuds “executors” pour exécuter les flows en parallèle

Déploiement :

- Docker Swarm (stack avec plusieurs replicas)
- Kubernetes (Deployment + Horizontal Pod Autoscaler)

Cette approche permet une **scalabilité horizontale**, en ajoutant dynamiquement des nœuds d’exécution selon la charge.

2. MinIO (stockage S3-like)

L’objectif est de garantir un stockage résilient et extensible.

Aujourd’hui : 1 instance MinIO, 1 disque.

Cible scalable :

- Cluster MinIO distribué (4+ nœuds) avec erasure coding

Avantages :

- tolérance à la panne disque / nœud
- montée en capacité en ajoutant des disques / nœuds

Usage :

- raw/ → données brutes
- processed/ → datasets transformés
- curated/ → datasets finaux

3. PostgreSQL (base analytique)

L’objectif est d’assurer la disponibilité et la performance des requêtes.

Aujourd’hui : 1 instance Postgres sur la VM.

Cible scalable :

- 1 nœud primaire
- 1 ou plusieurs nœuds réplicas (streaming replication)
- Failover automatique (Patroni / PgPool)

Séparation des usages :

- primaire → écritures (pipeline)
- réplicas → lectures (BI, Power BI)

À plus grande échelle, cette architecture peut évoluer vers un **système analytique dédié (data warehouse)** afin d'optimiser les performances de requêtes.

4. Couche d'exécution / infrastructure

L'infrastructure doit permettre d'isoler les rôles et d'assurer la résilience.

Option on-premise :

- Plusieurs VMs dédiées (Kestra, MinIO, Postgres)

Option cloud :

- Instances compute pour Kestra + executors
- Stockage MinIO ou S3 managé
- Postgres managé (RDS / Cloud SQL)
- Autoscaling sur CPU / nombre de jobs

5. Observabilité & supervision

La supervision garantit la stabilité et la réactivité de la plateforme.

- Dashboards Kestra
- Logs dans Postgres
- Prometheus + Grafana (CPU, RAM, I/O, latence DB)
- Alertes Slack déjà en place

10.3 Comment il faut le faire (approche par étapes)

Étape 1 – Séparer les rôles sur plusieurs services

Objectif : éviter un point de panne unique.

- VM / conteneur dédié pour Kestra
- VM / conteneur dédié pour MinIO
- VM / conteneur dédié pour Postgres

Étape 2 – Rendre chaque brique "HA"

- Postgres : réplication + failover
- MinIO : cluster distribué
- Kestra : mode server + executors

Étape 3 – Industrialiser le déploiement

- Docker Compose pour le dev
- Docker Swarm ou Kubernetes pour la prod
- Limites CPU / RAM par service

Étape 4 – Indicateurs de capacité

- Stockage : taille des buckets MinIO
- Compute : temps d'exécution des flows
- DB : temps de réponse, nombre de lignes

10.4 Coût de la scalabilité

Le tableau ci-dessous présente une estimation des coûts mensuels associés à une architecture scalable.

Composant	Besoin pour la scalabilité	Ressources nécessaires	Coût estimé (€/mois)	Commentaires
Kestra – Server	Nœud central (UI, API, scheduler)	2 vCPU, 4 Go RAM	**20–40 €**	Charge faible
Kestra – Executors	Exécution parallèle des flows	2–4 vCPU + 4–8 Go RAM par executor	**40–160 €**	Coût variable selon la charge
MinIO – Cluster distribué	Stockage scalable + tolérance aux pannes	4 nœuds × (2 vCPU, 4 Go RAM, 200 Go disque)	**80–200 €**	Poste principal du coût
PostgreSQL – HA	Haute disponibilité + réplication	2 nœuds × (2 vCPU, 4 Go RAM, 100 Go disque)	**60–120 €**	Réplica pour BI
Stockage additionnel	Historique + rétention 12 mois	+500 Go	**20–50 €**	Dépend du volume
Réseau interne	Communication Kestra ↔ MinIO ↔ Postgres	1–2 To/mois	**10–30 €**	Dépend du trafic
Observabilité	Prometheus + Grafana	1 vCPU, 2 Go RAM	**10–15 €**	Open source
Slack	Webhook + canal dédié	—	**0 €**	Slack gratuit
Support / maintenance	Optionnel	—	**0–200 €**	Selon support entreprise

Conclusion

Cette architecture scalable permet de garantir la performance, la disponibilité et la résilience du système tout en maîtrisant les coûts. Elle constitue une base solide pour une industrialisation future du pipeline de données.